# ARTIFICIAL INTELLIGENCE

## Assessment Tool – 2

### Constraint-Based Problem Solving

# 1. Hospital Doctor Assignment using Backtracking Search

## Given:

There are 3 doctors:

**D1, D2, D3**

There are 3 shifts:

**Morning, Afternoon, Night**

## Constraints:

1. D1 cannot work in Night shift.
2. D2 must work before D3.
3. Only one doctor can work in one shift.
4. D3 cannot work in Morning shift.
5. Every doctor should get exactly one shift.

The order of shifts is:

**Morning < Afternoon < Night**

---

## Step 1: Possible shifts

For D1:

**D1 = {Morning, Afternoon}**

because D1 cannot work at Night.

For D2:

**D2 = {Morning, Afternoon, Night}**

For D3:

**D3 = {Afternoon, Night}**

because D3 cannot work in Morning.

---

## Step 2: Apply Backtracking

First assign:

**D1 = Morning**

Check:

- D1 is not working Night → correct.
- No other doctor is assigned Morning yet → correct.

Now assign D2.

Try:

**D2 = Morning**

This is not possible because D1 is already in Morning.

So we backtrack.

Try:

**D2 = Afternoon**

Now:

**D1 = Morning**
**D2 = Afternoon**

Since D2 must work before D3, D3 must work after Afternoon.

Possible shift for D3 is Night.

So:

**D3 = Night**

---

## Constraint checking

| Constraint | Check | Result |
|---|---|---|
| D1 cannot work Night | D1 = Morning | Satisfied |
| D2 before D3 | Afternoon < Night | Satisfied |
| One doctor per shift | M, A, N are different | Satisfied |
| D3 cannot work Morning | D3 = Night | Satisfied |
| All doctors assigned once | D1, D2, D3 assigned | Satisfied |

Therefore, all constraints are satisfied.

## Final Schedule

**Morning → D1**

**Afternoon → D2**

**Night → D3**

## Backtracking sequence

D1 = Morning

↓

D2 = Morning → Not possible

↓

Backtrack

↓

D2 = Afternoon

↓

D3 = Morning → Not possible

↓

D3 = Afternoon → Already occupied

↓

D3 = Night → Valid

### Conclusion

Backtracking search tries one assignment at a time. If an assignment violates any constraint, it goes back and tries another assignment. In this problem, the final valid assignment is:

**D1 – Morning, D2 – Afternoon, D3 – Night.**

---

# 2. Robot Grid Problem using BFS

### Given:

The robot has to move from **Start (S)** to **Goal (G)** in a 5 × 5 grid.

The given constraints are:

1. The robot can move Up, Down, Left and Right.
2. Cost of every movement is 1.
3. The robot cannot move through obstacles.
4. Manhattan distance is given as the heuristic.

The question asks for node expansion, queue/priority updates, optimal path and cost.

### Note about the question

**The actual 5 × 5 grid is not provided in the question paper. Therefore, the exact path and exact cost cannot be calculated from the given information.**

So, to explain the BFS method, I am using the following **sample grid only as an illustration**.

---

# Sample Grid

Here:

**S = Start**
**G = Goal**
**X = Obstacle**

```
    1  2  3  4  5

1   S  .  X  .  .

2   X  .  X  .  X

3   .  .  .  .  .

4   X  X  X  .  X

5   .  .  .  .  G
```

Start:

**S = (1,1)**

Goal:

**G = (5,5)**

---

# BFS Method

BFS means **Breadth First Search**.

It visits the nodes level by level.

It uses a **queue**.

Initially:

Queue = [S]

---

## Step 1

Expand:

**(1,1)**

Possible movements:

- Up → outside grid
- Down → obstacle
- Left → outside grid
- Right → (1,2)

So:

Queue = [(1,2)]

---

## Step 2

Expand:

**(1,2)**

Possible new position:

**(2,2)**

So:

Queue = [(2,2)]

---

## Step 3

Expand:

**(2,2)**

Possible new position:

**(3,2)**

So:

Queue = [(3,2)]

---

## Step 4

Expand:

**(3,2)**

Possible positions:

- Left → (3,1)
- Right → (3,3)

So:

Queue = [(3,1), (3,3)]

---

## Step 5

Expand:

**(3,1)**

There is no useful unvisited path from this position.

So:

Queue = [(3,3)]

---

## Step 6

Expand:

**(3,3)**

Move right:

**(3,4)**

So:

Queue = [(3,4)]

---

## Step 7

Expand:

**(3,4)**

Possible positions:

- Up → (2,4)
- Right → (3,5)
- Down → (4,4)

So:

Queue = [(2,4), (3,5), (4,4)]

---

## Step 8

Expand:

**(2,4)**

Move up:

**(1,4)**

Queue becomes:

Queue = [(3,5), (4,4), (1,4)]

---

## Step 9

Expand:

**(3,5)**

No useful new position is added.

Queue = [(4,4), (1,4)]

---

### Step 10

Expand:

**(4,4)**

Move down:

**(5,4)**

Queue:

Queue = [(1,4), (5,4)]

---

### Step 11

Expand:

**(1,4)**

Move right:

**(1,5)**

Queue:

Queue = [(5,4), (1,5)]

---

### Step 12

Expand:

**(5,4)**

Move right:

**(5,5) = G**

Therefore, the goal is reached.

---

## Optimal Path

The path obtained is:

(1,1)
↓
(1,2)
↓
(2,2)
↓
(3,2)
→
(3,3)
→
(3,4)
↓
(4,4)
↓
(5,4)
→
(5,5)

Therefore:

S → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4) → (5,4) → G

Number of moves = 8

Cost of each move = 1

Therefore:

Total cost = 8 × 1 = 8

### Answer for the sample grid:

Optimal path cost = 8

---

# Manhattan Distance

The Manhattan distance is calculated as:

**h(n) = |x - xg| + |y - yg|**

For example, for node **(3,4)** and goal **(5,5)**:

**h(n) = |3 - 5| + |4 - 5|**

**= 2 + 1**

**= 3**

Manhattan distance gives the approximate distance between the current cell and the goal.

### Note

Normally, BFS uses a normal **FIFO queue** and does not use a heuristic. The question also mentions Manhattan distance and priority queue, which are more commonly used in heuristic or cost-based searches. Therefore, for this answer, BFS is treated as the main search algorithm because that is what the question specifically asks.

### Conclusion

BFS explores the grid level by level and finds the shortest path when every movement has the same cost. In the sample grid, the shortest path has 8 movements and the total cost is 8.

---

# 3. Autonomous Rescue Robot

## a) Analysis of the Constraints

The rescue robot is working in a disaster-affected building. Its main aim is to reach the survivor at **G**.

The robot has the following constraints:

1. It can move Up, Down, Left and Right.
2. Some cells are blocked.
3. It can only see nearby cells.
4. Normal movement costs 1.
5. Entering a risky zone has an additional cost of 2.
6. It has to minimize the total cost.
7. It should navigate safely.

8. It has to update its knowledge while moving.

These constraints affect the robot's decisions because it does not know the complete environment from the beginning.

For example, if the robot reaches a new room and finds an obstacle, it has to change its plan and find another path.

---

# Effect of the constraints

## Movement

The robot can move only:

**Up, Down, Left and Right.**

It cannot move diagonally.

## Obstacles

The robot cannot enter blocked cells.

## Limited sensing

The robot can see only adjacent cells. Therefore, it does not know the complete map.

## Normal movement cost

Every normal movement has:

**Cost = 1**

## Risky zone

A risky zone has an additional cost of 2.

Therefore:

**Risky movement cost = 1 + 2 = 3**

## Minimum cost

The robot should choose a path with minimum total cost.

It should not simply choose the path with the least number of movements.

---

# b) Solution using Uniform Cost Search and Online Search

For this problem, **Uniform Cost Search (UCS)** together with an **Online Search Agent** can be used.

Uniform Cost Search selects the position having the lowest total cost from the starting position.

Let:

**g(n) = cost from Start to current node**

For a normal cell:

**Cost = 1**

For a risky cell:

**Cost = 3**

---

## Steps followed by the robot

### Step 1 – Start

The robot starts from:

**S**

Initial cost:

**g(S) = 0**

---

### Step 2 – Observe

The robot checks the cells around its current position.

It identifies:

- Free cells

- Obstacles
- Risky cells
- Goal

---

## Step 3 – Calculate cost

The robot calculates the cost of reaching each possible next cell.

Example:

Normal cell → cost 1

Risky cell → cost 3

Blocked cell → cannot enter

---

## Step 4 – Choose the lowest-cost path

UCS chooses the position with the lowest accumulated cost.

For example:

Path A = cost 4
Path B = cost 6
Path C = cost 5

The robot chooses **Path A**.

---

## Step 5 – Move and observe again

After moving, the robot checks its nearby cells again.

It may discover a new obstacle or risky area.

Then it updates its knowledge.

---

## Step 6 – Replan

If the original path is blocked, the robot searches for another path.

The process is:

Observe

↓

Update knowledge

↓

Calculate costs

↓

Choose lowest-cost path

↓

Move

↓

Observe again

↓

Replan if required

This process continues until the robot reaches the survivor.

---

# c) AI Concepts Used

## 1. Intelligent Agent

An intelligent agent is a system that observes its environment and takes actions to achieve a goal.

The rescue robot is an intelligent agent because it:

- Observes the environment.
- Takes decisions.
- Moves according to those decisions.
- Updates its knowledge.
- Tries to reach the survivor safely.

So the robot follows:

**Perceive → Decide → Act**

---

# 2. Rationality

A rational agent chooses the best action based on the information available to it.

In this problem, the robot should:

- Avoid obstacles.
- Avoid unnecessary risky areas.
- Minimize total cost.
- Update its knowledge.
- Change its route when required.
- Reach the survivor safely.

Therefore, the robot behaves rationally when it chooses the **best safe and low-cost action based on its current information**.

---

# 3. Environment Type

The environment can be classified as follows:

## Partially Observable

The robot can observe only nearby cells.

Therefore, it cannot see the complete building.

**Environment = Partially observable**

## Dynamic

The robot may discover new obstacles or changes while moving.

**Environment = Dynamic**

## Sequential

One action affects the next action.

For example, choosing a risky path increases the total cost.

**Environment = Sequential**

## Uncertain

The robot does not know the complete environment initially.

**Environment = Uncertain**

### Discrete

The building is represented using separate grid cells and the robot makes individual movements.

**Environment = Discrete**

---

## Environment Classification

| Property | Type |
|---|---|
| Observability | Partially observable |
| Environment | Dynamic |
| Decision process | Sequential |
| Information | Uncertain / incomplete |
| Representation | Discrete |

---

# d) Why Uniform Cost Search is Suitable

Uniform Cost Search is suitable because all movements do not necessarily have the same total cost.

A normal movement costs:

1

A risky movement costs:

1 + 2 = 3

For example:

### Path A

5 normal movements:

**Cost = 5 × 1 = 5**

### Path B

3 normal movements and 1 risky movement:

**Cost = (3 × 1) + 3**

**= 6**

Even though Path B uses fewer movements, Path A is cheaper.

Therefore, the robot should compare the **total path cost** instead of only counting the number of movements.

UCS is useful because it always expands the lowest-cost path first.

---

## Why Online Search is also required

The robot has limited sensing.

It cannot know the complete building before starting.

Therefore, it has to learn while moving.

For example:

Start
↓
Observe
↓
Move
↓
Observe new area
↓
Find obstacle

↓

Update map

↓

Find another path

↓

Continue

↓

Reach survivor

This makes online search suitable for the rescue robot.

---

# Final Conclusion

The rescue robot is an **intelligent agent** because it observes the environment, makes decisions and performs actions to reach the survivor.

The environment is **partially observable, dynamic, sequential, uncertain and discrete**.

Since different cells can have different costs, **Uniform Cost Search** is suitable for finding a low-cost path. Since the robot does not know the complete environment beforehand, it should also use **online search**, where it observes, updates its knowledge and replans during navigation.

Thus, the combination of an **intelligent agent, rational decision-making, Uniform Cost Search and online search** is suitable for solving this rescue problem.